

Practice Questions Solutions

Q1. What is a Data Structure? Give 3 real-life examples.

Definition: A data structure is a specialized format used to organize, manage, process, and store data efficiently in a computer's memory so that operations can be executed quickly.

3 Real-Life Examples:

- **A Stack of Plates:** You can only place a new plate on the top, and you must remove the top plate first. This mimics a **Stack (LIFO)** data structure.
- **A Queue at a Ticket Counter:** The first person standing in line is served first. This mimics a **Queue (FIFO)** data structure.
- **A Dictionary or Phone Book:** Information is organized systematically (e.g., alphabetically) for quick lookups, closely mimicking **Trees or Hash Tables**.

Q2. Differentiate between Linear and Non-linear Data Structures.

Feature	Linear Data Structure	Non-linear Data Structure
Arrangement	Elements are arranged sequentially or linearly.	Elements are arranged hierarchically or interconnectedly.
Traversal	Can be completely traversed in a single run.	Requires multiple runs or complex algorithms (like BFS/DFS) to traverse.
Levels	Single-level structure.	Multi-level structure.
Memory Utilization	Can be less efficient (e.g., static memory in arrays).	Highly memory efficient.
Examples	Array, Linked List, Stack, Queue.	Tree, Graph.

Q3. Explain the internal structure of an Array with a memory diagram.

An **Array** is a collection of items of the same data type stored at contiguous (continuous) memory locations.

- **Index:** Starts at **0** and goes up to ***n-1*** (where ***n*** is the size).
- **Base Address:** The memory location of the very first element.
- **Element Size:** Every element takes a uniform amount of memory (e.g., 4 bytes for an integer).

Memory Diagram:

Let's take an integer array `int arr[4] = {10, 20, 30, 40};`. Assuming the base address is 1000 and each integer takes 4 bytes:

Index:	[0]	[1]	[2]	[3]
Value:	10	20	30	40
Memory Addr:	1000	1004	1008	1012
	^			
	Base Address			

Q4. Write the algorithm and C program for Array insertion.

Algorithm:

- 1. Start.
- 2. Loop backward from the last element down to the target position and shift each element one slot to the right.
- 3. Insert the new element at the targeted position.
- 4. Increase the overall size of the array by 1.
- 5. Stop.

C Program:

```
#include <stdio.h>

int main() {
    int arr[100] = {10, 20, 30, 40, 50};
    int size = 5;
    int element = 25, position = 2; // Insert 25 at index 2

    // Shift elements to the right
    for (int i = size - 1; i >= position; i--) {
        arr[i + 1] = arr[i];
    }

    // Insert element
    arr[position] = element;
    size++;

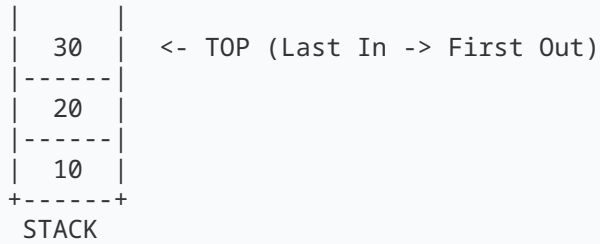
    // Print updated array
    printf("Array after insertion: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Q5. Explain Stack using LIFO concept with a diagram.

A **Stack** is a linear data structure following the **LIFO (Last In, First Out)** principle. The last item added to the stack is the very first one to be removed.

- **Push:** Operation to insert an element.
- **Pop:** Operation to remove an element.
- **Top:** The pointer pointing to the topmost element.

Diagram:



Q6. What is the difference between Stack and Queue?

Feature	Stack	Queue
Working Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)
Access Points	Operations (Push/Pop) happen at only one end (Top).	Insertion happens at the Rear ; Deletion happens at the Front .
Pointers Used	Uses a single pointer: top.	Uses two pointers: front and rear.
Real-life analogy	A stack of dinner plates.	A line of people waiting for a bus.

Q7. Write C code for Enqueue and Dequeue operations.

Below is an array-based implementation of a Queue:

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

// Enqueue operation (Insertion)
void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Queue Overflow!\n");
    } else {
        if (front == -1) front = 0;
        rear++;
        queue[rear] = value;
        printf("Enqueued: %d\n", value);
    }
}

// Dequeue operation (Deletion)
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow!\n");
    } else {
        printf("Dequeued: %d\n", queue[front]);
        front++;
        if (front > rear) { // Reset queue when empty
            front = rear = -1;
        }
    }
}
```

Q8. Explain Singly Linked List with a node diagram.

A **Singly Linked List** is a linear data structure where elements are not stored in contiguous memory spots. Instead, elements are stored in independent blocks called **Nodes**, linked together via pointers.

Each node contains two components:

- **Data:** Holds the actual element value.
- **Next Pointer:** Stores the memory address pointing to the next node. The last node points to **NULL**.

Node Diagram:

